

1. Java Operator Precedence

Operator precedence defines the order in which operators are evaluated in an expression.

Higher precedence operators are evaluated first.

Precedence	Operator	Associativity	Category
1	<code>() [] .</code>	Left to Right	Parentheses, array access, member access
2	<code>++ -- + - ~ !</code>	Right to Left	Unary (postfix, prefix, logical NOT, bitwise NOT)
3	<code>* / %</code>	Left to Right	Multiplication, Division, Modulus
4	<code>+ -</code>	Left to Right	Addition, Subtraction
5	<code><< >> >>></code>	Left to Right	Bitwise Shift
6	<code>< <= > >= instanceof</code>	Left to Right	Relational
7	<code>== !=</code>	Left to Right	Equality
8	<code>&</code>	Left to Right	Bitwise AND
9	<code>^</code>	Left to Right	Bitwise XOR
10	<code>`</code>	<code>`</code>	Left to Right
11	<code>&&</code>	Left to Right	Logical AND
12	<code>`</code>		<code>`</code>
13	<code>? :</code>	Right to Left	Ternary Operator
14	<code>`=, +=, -=, *=, /=, %=, &=,</code>	<code>=, ^=, <<=, >>=, >>>=</code>	Right to Left

Operator Precedence Example

```
public class OperatorPrecedence {  
    public static void main(String[] args) {  
        int result = 10 + 5 * 2; // Multiplication (*) has higher precedence than add.  
        System.out.println(result); // Output: 20  
  
        int x = 5, y = 10, z = 15;  
        int answer = x + y * z / x - y; // (*, / have higher precedence than +, -)  
        System.out.println(answer); // Output: 10  
    }  
}
```

2. Java Bit Manipulation Operations

Bit manipulation involves operating on binary representations of numbers.

Operator	Description	Example (a=5, b=3)	Result (Binary)	Result (Decimal)
&	Bitwise AND	5 & 3	0101 & 0011	0001 → 1
	Bitwise OR	5 3	0101 0011	0111 → 7
^	Bitwise XOR	5 ^ 3	0101 ^ 0011	0110 → 6
~	Bitwise Complement	~5	~0101	1010 → -6
<<	Left Shift	5 << 1	0101 << 1	1010 → 10
>>	Right Shift (Sign-preserving)	5 >> 1	0101 >> 1	0010 → 2
>>>	Unsigned Right Shift	-5 >>> 1	1111...1011 >>> 1	0111...1101 (Positive)

3. Bitwise AND (&)

Use Case: Masking & Checking Bits

```
public class BitwiseAND {  
    public static void main(String[] args) {  
        int a = 5, b = 3; // 5 = 0101, 3 = 0011  
        System.out.println(a & b); // Output: 1 (0001)  
    }  
}
```

- ◆ Used to check if a bit is set (e.g. `if ((num & 1) == 0)` checks if `num` is even).
-

4. Bitwise OR (|)

Use Case: Setting a bit

```
public class BitwiseOR {  
    public static void main(String[] args) {  
        int a = 5, b = 3; // 5 = 0101, 3 = 0011  
        System.out.println(a | b); // Output: 7 (0111)  
    }  
}
```

- ◆ Used to set specific bits in a number.
-

5. Bitwise XOR (^)

Use Case: Swapping Two Numbers Without a Temporary Variable

```
public class BitwiseXORSwap {  
    public static void main(String[] args) {  
        int a = 5, b = 3;  
        a = a ^ b; // a = 6 (0101 ^ 0011 = 0110)  
        b = a ^ b; // b = 5  
        a = a ^ b; // a = 3  
        System.out.println("a: " + a + ", b: " + b); // a=3, b=5  
    }  
}
```

- ♦ XOR returns 1 when bits are different → Useful for toggling bits.
-

6. Bitwise NOT (~)

Use Case: Getting Two's Complement (Negation)

```
public class BitwiseNOT {  
    public static void main(String[] args) {  
        int a = 5;  
        System.out.println(~a); // Output: -6  
    }  
}
```

- ♦ Flips all bits: ~5 is -6 (because of two's complement representation).
-

7. Left Shift (<<)

Use Case: Multiply by Powers of 2

```
public class LeftShift {  
    public static void main(String[] args) {  
        int a = 5;  
        System.out.println(a << 1); // Output: 10 (5 * 2)  
        System.out.println(a << 2); // Output: 20 (5 * 4)  
    }  
}
```

- ◆ Each left shift doubles the number.
-

8. Right Shift (>>)

Use Case: Divide by Powers of 2

```
public class RightShift {  
    public static void main(String[] args) {  
        int a = 20;  
        System.out.println(a >> 1); // Output: 10 (20 / 2)  
        System.out.println(a >> 2); // Output: 5 (20 / 4)  
    }  
}
```

- ◆ Each right shift halves the number.
-

9. Unsigned Right Shift (>>>)

Use Case: Shift Bits Right Without Sign Extension

```
public class UnsignedRightShift {  
    public static void main(String[] args) {  
        int a = -10;  
        System.out.println(a >>> 1); // Shifts right and fills leftmost bit with 0  
    }  
}
```

- ◆ Unlike >>, it does NOT preserve the sign bit (fills with 0 instead).
-

10. Checking if a Number is Power of Two

```
public class PowerOfTwo {  
    public static boolean isPowerOfTwo(int n) {  
        return (n > 0) && (n & (n - 1)) == 0;  
    }  
  
    public static void main(String[] args) {  
        System.out.println(isPowerOfTwo(16)); // true  
        System.out.println(isPowerOfTwo(18)); // false  
    }  
}
```

- ◆ Numbers that are powers of two have only one 1 bit in their binary representation.
-

Summary

Bitwise Operation	Use Case
& (AND)	Checking if a bit is set
^ (XOR)	Swapping numbers, toggling bits
~ (NOT)	Two's complement (negation)
<< (Left Shift)	Multiplying by 2
>> (Right Shift)	Dividing by 2
>>> (Unsigned Right Shift)	Shift right without sign extension
